Parallel GETH project summary

Submitted by: Itay Elam, 05.04.2026

General implementation:

- Using the declared read/write sets (EIP-2930 assumption), we divide all block transactions into waves. A wave consists of transactions that can be parallelized as they do not have any dependency between them (this is not entirely correct, more on that in future work section).
- Each wave gets a copy of the global StateDB (which handles all the data related structures that are relevant for the transaction). The transactions then work concurrently and when they finish, they are joined together.
- Next we merge their states as much as possible into the true global state as this requires in-order merging meaning if we have $wave_1 = \{tx_1, tx_3\}, wave_2 = \{tx_2\}$, we will merge only the results from tx_1 into the global stateDB from which we will copy for tx_2. This is done as the merging order of receipts and logs is important. Doing so does not hinder future correctness as in serial execution tx_2 does not expect the results from tx_3 to be present, it only has higher memory consumption with the current full stateDB cloning implementation.

Implementation details:

Parallel execution model (core/state_processor.go)

- When ParallelTxGroupingByStorageOverlap is on, reorder independent transactions into waves. When ParallelTxWaveExecution is on and a wave has more than one transaction, each transaction runs in parallel with its own StateDB.Copy(), with SetDeferTrieFlush(true) (detailed in the next section) its own vm.EVM, and a shared atomic GasPool.
- After Wait, results are merged back in strict ascending transaction index via MergeParallelChildInto.
- We treat the main Go process as the parent and the subroutines as children.

StateDB fork + merge (core/state/statedb.go)

- deferTrieFlush: on forked DBs, IntermediateRoot only runs Finalise and returns an empty root (no trie flush); Commit returns originalRoot without persisting.

- Finalise still drives transaction boundary accounting. when deferTrieFlush is set, it records parallelMergeAddrs from journal.dirties (only when non-empty) before clearing the journal because clearJournalAndRefund() wipes journal.dirties right after Finalise, and that map is the only compact record of which accounts were actually touched during the transaction, without copying those addresses first, MergeParallelChildInto would not know which stateObjects and mutations to apply onto the parent, so it would either miss updates (leave storage/balances unchanged) or have to merge the entire copied stateObjects map (risky because the child is a full snapshot of the pre-wave state and includes many accounts the transaction never modified).

  *Recording only when len(journal.dirties) > 0 avoids rewriting on a previously valid parallelMergeAddrs when a second Finalise with an empty journal (e.g. if IntermediateRoot runs again on a defer child) – this is a guardrail against internal Geth implementation nuances.

- MergeParallelChildInto copies touched accounts from child to parent, applies markUpdate / markDelete, merges logs (with log index reassignment to match parent.logSize).

Tests (eth/parallel_vm_block_test.go)

- After InsertChain, logs receipt status, gas used, and cumulative gas for the head block (helps separate OOG / failed transactions from merge bugs).
- Control over gas limit and per-transaction gas via constants so heavy rounds in isolatedJob can run.
- Consist of a test with completely independent transactions, a test with completely dependent transactions and a test with a mixture of dependent and independent transactions where some are parallelizable and some are not.

Project Result

- Successfully runs concurrent transactions.
- Removes the shared StateDB + concurrent journal/snapshot panic errors for parallel waves.
- Correctness is Dependent on:
    - Every transaction declares a full read write set (not just cold starts but every address for the whole contract).

    - Every transaction in the wave requires the same pre-wave state (should be guaranteed from the above)
    - No accesses to block.coinbase
    - Gas is not exhausted
    - Contracts don't fail

- Speedup results from running each experiment three times and averaging the results. Ran on 12 Core CPU, all 200 transactions are launched at once in parallel execution (all in one wave) – probably over saturating the CPU:

Average seconds per transaction	serial average total time (s)	parallel average total time (s)	Average speedup
0.01037288367	2.074576733	0.4288112333	4.837971984
0.04957299767	9.914599533	4.0072946	2.474137922
0.113482577	23.54695957	9.400588967	2.504838755
0.2040723702	40.81447403	16.64105027	2.452638108

Future work for finer-grained / better parallelism

1. Cheaper isolation than full Copy(): scoped journals, or copy-on-write only for touched accounts to cut memory and CPU.
2. Finer grouping than declared addresses: Use storage-slot overlap, static/dynamic read/write sets, or profiling to build larger safe waves without relying only on EIP-2930 lists.
3. Correct cross-transaction dependencies: Model coinbase, system contracts, and any account updated by earlier transactions in the block.
5. Error recovery: Conflict detection at receipt merge time (or fallback to serial) instead of silent wrong state.

Assumptions we're making:

- Declared access lists: we treat address-disjoint groups as safe to run from the same pre-wave snapshot. Incomplete lists or hidden dependencies (e.g. coinbase) break that.
- Same baseline for every transaction in a wave matches sequential semantics only when no transaction in the wave depends on state produced by a skipped lower index in that block.
- Merge commutativity: disjoint writes to accounts/slots should compose like serial application